

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	50% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	C Nithisha	Reg. No.:	192424375
Section / Batch:	2024	Date:	03-08-2026

Code of Conduct

I C Nithisha 192424375 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student C Nithisha

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

Apply rules to decompose each word into root and suffix components.

Differentiate between inflectional and derivational forms within the dataset.

Normalize all variants to a common base form for efficient retrieval.

Determine the parsed structure and normalized output for each word.

Answer:

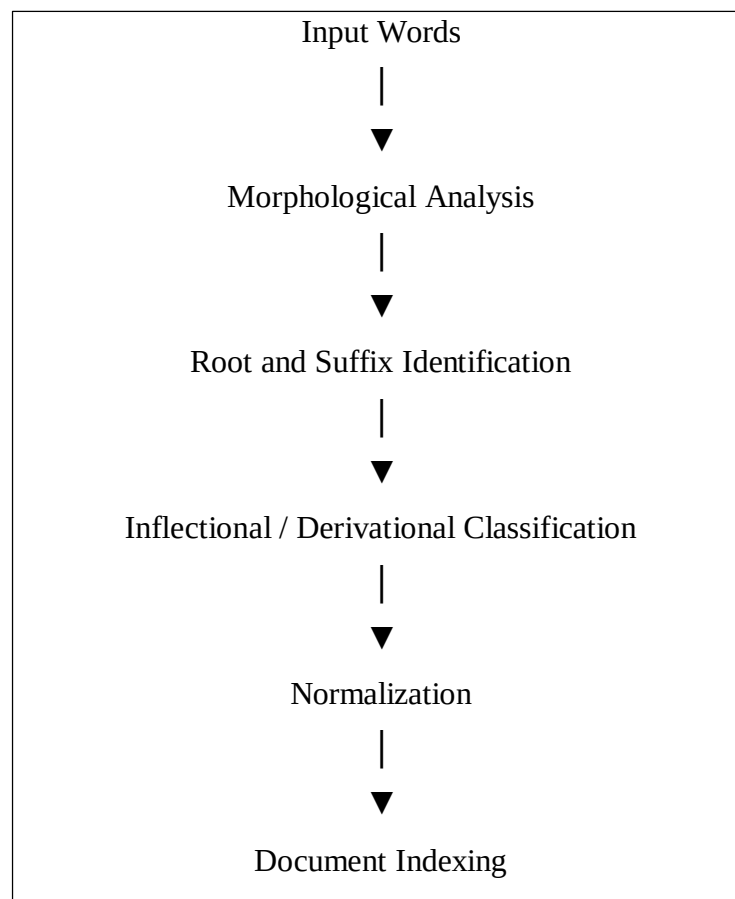
Given Input Words

connected, connecting, connection

Design a Morphological Analysis Pipeline for a Document Indexing System

A morphological analysis pipeline identifies the root word and separates suffixes from different word forms. It then classifies the words as inflectional or derivational and normalizes them into a common base form. This helps a document indexing system retrieve all related documents using a single index term.

Morphological Analysis Pipeline



Rule-Based Decomposition

Rule 1

If a word ends with -ed, remove -ed to obtain the root.

Example:

connected = connect + ed

Rule 2

If a word ends with -ing, remove -ing to obtain the root.

Example:

connecting = connect + ing

Rule 3

If a word ends with -ion, identify the base verb by removing the derivational suffix.

Example:

connection = connect + ion

Morphological Decomposition

Input Word	Root	Suffix
Connected	connect	ed
connecting	connect	ing
connection	connect	ion

Inflectional and Derivational Forms

Word	Type	Reason
connected	Inflectional	Indicates past tense; the meaning of the root remains the same.
connecting	Inflectional	Indicates continuous action; the meaning of the root remains the same.
connection	Derivational	Forms a noun from the verb "connect".

Normalization

All word variants are normalized to the common base form:

connect

Parsed Structure

connected

└── Root : connect

└── Suffix : ed

connecting

└── Root : connect

└── Suffix : ing

connection

└─ Root : connect

└─ Suffix : ion

Normalized Output

Input Word	Normalized Output
connected	connect
connecting	connect
connection	connect

Conclusion

The morphological analysis pipeline successfully decomposes each input word into its root and suffix, classifies the words as inflectional or derivational, and normalizes all variants to the common base form connect. This normalization improves document indexing and enables efficient information retrieval by treating related word forms as a single searchable term.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

Identify prefixes, suffixes, and base forms using rule-based decomposition.

Ensure semantic consistency across derived word forms.

Classify each transformation as inflectional or derivational.

Produce the root and morphological breakdown for each word.

Answer:

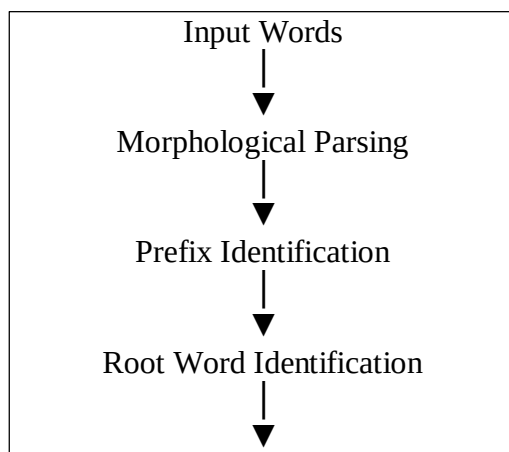
Given Input Words

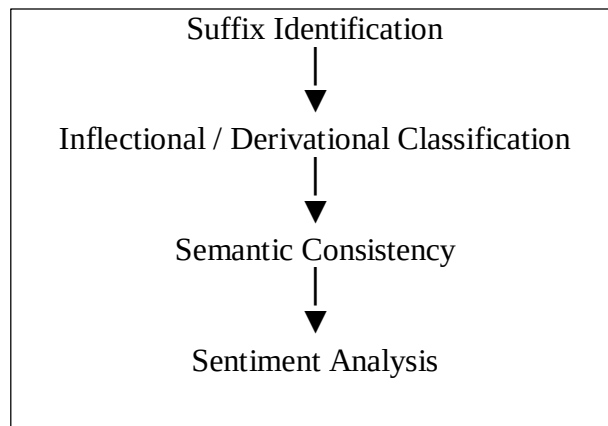
unhappy, happiness, happily

Design a Morphological Parsing Module for Sentiment Analysis

A morphological parsing module identifies the internal structure of words by separating prefixes, suffixes, and the base form. In sentiment analysis, this helps recognize that different word forms are related to the same root concept, improving the consistency of sentiment interpretation.

Morphological Parsing Module





Rule-Based Decomposition

Rule 1

If a word begins with un-, identify un- as the prefix and remove it to obtain the base word.

Example:

unhappy = un + happy

Rule 2

If a word ends with -ness, identify -ness as the suffix and remove it to obtain the base word.

Example:

happiness = happy + ness

Rule 3

If a word ends with -ly, identify -ly as the suffix and remove it to obtain the base word.

Example:

happily = happy + l

Morphological Breakdown

Input Word	Prefix	Root	Suffix
unhappy	un	happy	—
happiness	—	happy	ness
happily	—	happy	ly

Semantic Consistency

Although the words have different forms, they all share the common root happy.

- unhappy expresses the opposite meaning of *happy*.
- happiness represents the state or feeling of being happy.
- happily describes performing an action in a happy manner.

The parser recognizes these relationships so that the sentiment analysis system can correctly interpret the meaning of each word.

Inflectional and Derivational Classification

Word	Type	Reason
unhappy	Derivational Prefix	un- changes the meaning of the word.
happiness	Derivational Suffix	-ness changes the adjective into a noun.
happily	Derivational Suffix	-ly changes the adjective into an adverb.

Parsed Structure

unhappy

└─ Prefix : un
└─ Root : happy
└─ Suffix : —

happiness

└─ Prefix : —
└─ Root : happy
└─ Suffix : ness

happily

└─ Prefix : —
└─ Root : happy
└─ Suffix : ly

Final Morphological Output

Input Word Morphological Breakdown Root

unhappy	un + happy	happy
happiness	happy + ness	happy
happily	happy + ly	happy

Conclusion

The morphological parsing module successfully identifies the prefixes, suffixes, and root forms of the given words. It classifies all transformations as derivational, since each transformation creates a new word or changes its meaning or grammatical category. The common root happy ensures semantic consistency, which improves the accuracy of sentiment analysis by relating different word forms to the same base concept.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

Apply morphological rules to strip affixes and identify the base form.

Handle both grammatical variations and word formation changes.

Ensure consistent root extraction across all forms.

Determine the stem and classification for each input word.

Answer:

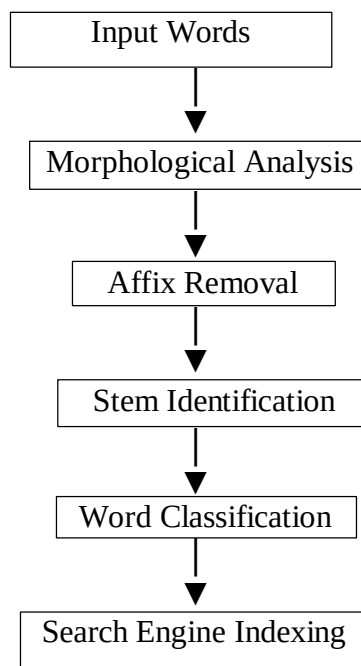
Given Input Words

played, player, playing

Design a Stemming-Based Preprocessing Module for a Search Engine

A stemming-based preprocessing module removes prefixes or suffixes from words to identify their common stem. In a search engine, stemming helps retrieve relevant documents by treating different forms of a word as the same term.

Stemming-Based Preprocessing Module



Morphological Rules

Rule 1

If a word ends with -ed, remove -ed to obtain the stem.

Example:

played = play + ed

Rule 2

If a word ends with -er, remove -er to obtain the stem.

Example:

player = play + er

Rule 3

If a word ends with -ing, remove -ing to obtain the stem.

Example:

playing = play + ing

Morphological Breakdown

Input Word Stem Removed Affix

played play ed

player play er

playing play ing

Handling Grammatical Variations and Word Formation

- played represents the past tense of the verb play.
- playing represents the present participle/continuous form of the verb play.
- player is a noun formed from the verb play.

Although the words differ in grammatical form and word formation, they all share the same stem **play**.

Classification

Word Classification Reason

played Inflectional Indicates past tense without changing the core meaning.

player Derivational Forms a noun from the verb play.

playing Inflectional Indicates continuous action without changing the core meaning.

Consistent Root Extraction

All input words are reduced to the common stem:

play

This ensures that a search engine indexes all three words under the same base term, improving search efficiency.

Final Output

Input Word Stem Classification

played play Inflectional

player play Derivational

playing play Inflectional

Conclusion

The stemming-based preprocessing module successfully removes the suffixes from the given words and extracts the common stem play. It correctly handles both grammatical variations (played, playing) and word formation (player) while ensuring consistent root extraction. This improves search engine indexing and allows documents containing different forms of the same word to be retrieved together.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

Apply state transitions to represent suffix transformations and irregular forms in verb inflection.

Differentiate between regular and irregular inflectional patterns within the dataset.

Ensure consistent mapping of all forms to a common root representation.

Determine the state transitions, morphological breakdown, and normalized output for each word.

Answer:

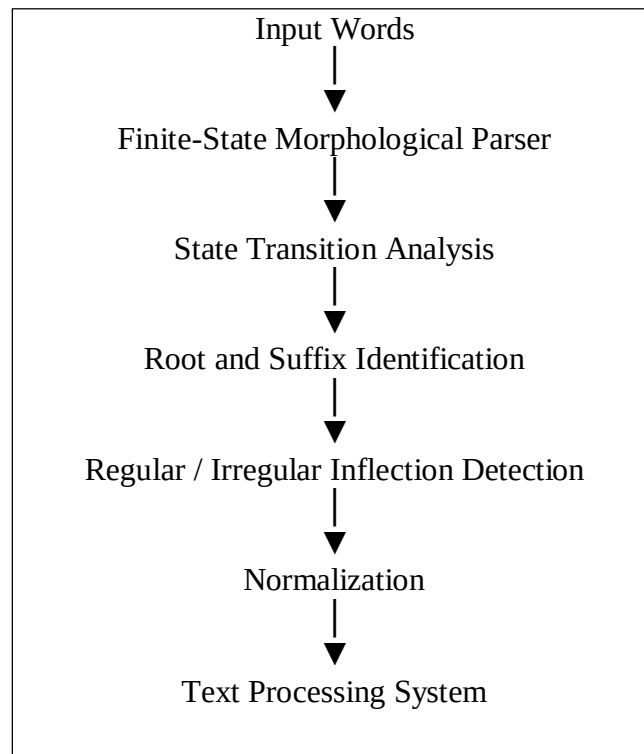
Given Input Words

writes, writing, written

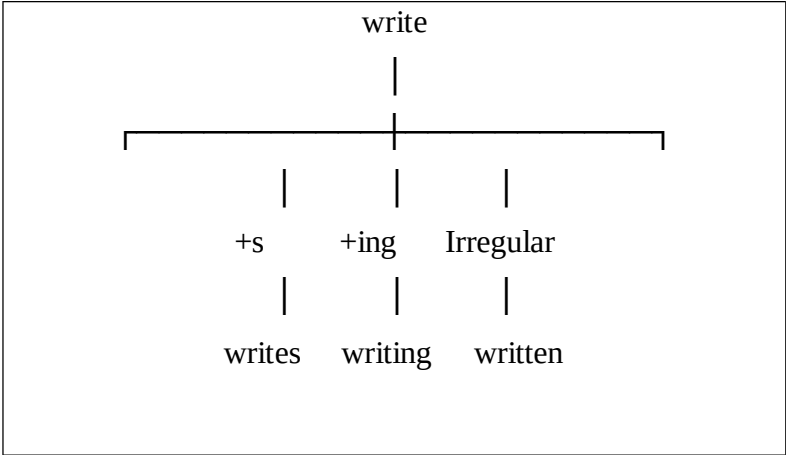
Design a Finite-State Morphological Parsing Module for a Text Processing System

A finite-state morphological parser analyzes words by applying state transitions to identify the root word and its inflectional forms. It handles both regular and irregular word transformations and maps all word forms to a common root representation for efficient text processing.

Finite-State Morphological Parsing Module



State Transitions



Morphological Breakdown

Input Word	Root	Suffix / Form	Pattern
writes	write	s	Regular Inflection
writing	write	ing	Regular Inflection
written	write	en (irregular form)	Irregular Inflection

Regular and Irregular Inflection

Word	Inflection Type	Explanation
writes	Regular	The suffix -s indicates the third-person singular present tense.
writing	Regular	The suffix -ing indicates the present participle or continuous form.
written	Irregular	The past participle is formed irregularly and does not follow the standard -ed pattern.

Common Root Representation

All word forms are mapped to the common root:

write

This normalization ensures that different forms of the same verb are treated as one term during text processing

Parsed Structure

writes

└─ Root : write
└─ Suffix : s

writing

└─ Root : write

└─ Suffix : ing

written

└─ Root : write

└─ Irregular Past Participle : en

Normalized Output

Input Word Normalized Output

writes write

writing write

written write

Conclusion

The finite-state morphological parsing module successfully analyzes the words writes, writing, and written using state transitions. It distinguishes between regular inflectional forms (writes, writing) and the irregular inflectional form (written). Finally, it normalizes all words to the common root write, improving consistency and efficiency in text processing systems.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.

Handle variations in suffix patterns while preserving semantic consistency.

Ensure uniform root extraction across all derived forms.

Determine the stemming steps, intermediate forms, and final stem for each input word.

Answer:

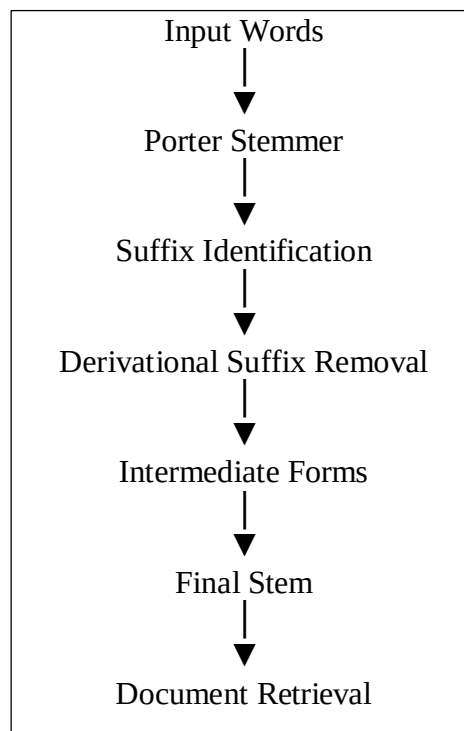
Given Input Words

relational, relation, relate

Design a Porter Stemmer-Based Preprocessing Module for Document Retrieval

A Porter Stemmer-based preprocessing module removes derivational suffixes from words to obtain a common stem. This process groups related word forms together, improving the efficiency and accuracy of document retrieval systems.

Porter Stemmer-Based Preprocessing Module



Porter Stemming Rules

Rule 1

If the word ends with -ational, replace it with -ate.

Example:

relational → relate

Rule 2

If the word ends with -ion, remove the suffix to obtain the base form.

Example:

relation → relate

Rule 3

Remove the final -e to obtain the final stem.

Example:

relate → relat

Step-by-Step Stemming

Input Word	Step 1	Step 2	Final Stem
relational	relate	Remove e	relat
relation	relate	Remove e	relat
relate	Remove e	—	relat

Intermediate Forms

Input Word Intermediate Form Final Stem

relational	relate	relat
relation	relate	relat
relate	relate	relat

Uniform Root Extraction

All input words are reduced to the common stem:

relat

This ensures that all related words are indexed under the same stem, improving document retrieval performance.

Final Output

Input Word Stemming Steps Final Stem

relational	relational → relate → relat	relat
relation	relation → relate → relat	relat
relate	relate → relat	relat

Conclusion

The Porter Stemmer-based preprocessing module successfully removes derivational suffixes and extracts the common stem **relat** from all input words. The intermediate stemming steps preserve semantic consistency while ensuring uniform root extraction. This common stem improves document retrieval by allowing different word forms to be indexed and searched together.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____